Charon: Runtime Boundary Enforcement for Autonomous Agents

Working Paper, Revision 0.1, June 2026

Charon Protocol
github.com/CharonAI-code/charon
x.com/Charon_AI

Abstract. Autonomous agents increasingly cross the boundary between language and execution: they request commands, read files, call tools, touch environment variables, and initiate side effects. Controls placed only inside the prompt are soft, while logs written after execution arrive too late. **Charon** introduces a local runtime boundary for agent actions. An agent may request an action, but the action is represented as data and evaluated before it reaches the operating system, a tool, a file, a network target, or a secret-bearing environment. Charon returns one of three verdicts: PASS, PAUSE, or DENY. Pass permits execution, PAUSE places the action in a review queue, and DENY blocks it before launch. Each decision produces a receipt binding the action, verdict, policy hash, boundary trace, redactions, timestamps, and local identity proof. This paper frames agent security as runtime boundary enforcement rather than prompt hardening: the attempted action, not the model's internal reasoning, is the governable unit.

1. Introduction

Agents turn model outputs into operations. The important security question is no longer only whether a response is safe, but whether a requested action should be allowed to cross into the machine boundary. Charon is designed for that crossing.

The gap exists because model reasoning and machine execution have different failure modes. A model can summarize an issue, read a web page, inspect a tool description, or retrieve a file and then synthesize an action from that mixed context. The action itself, however, is not probabilistic once launched. A shell command either runs or does not run; a file is either read or not read; a token is either exposed or withheld; an API call either mutates external state or it does not. The decisive security boundary is therefore the translation from model output into executable action.

Prompt instructions are useful, but they are not an enforcement mechanism. They rely on the same model that is interpreting untrusted context to also remember and obey the security policy. That coupling is fragile. An agent can be induced to reinterpret a task, follow instructions embedded in retrieved content, select a misleading tool, or perform a dangerous operation while producing a harmless-looking final response. If the only hard control is a log written afterward, the side effect has already occurred.

1.1 Thesis

Charon treats the attempted action as the unit of control. Before execution, the action is normalized and evaluated against structured policy: command rules, file boundaries, environment exposure, network targets, output redaction, and review requirements. The result is not advice to the model, but a verdict at the runtime boundary. A permitted action can execute; a reviewable action is queued; a forbidden action is denied before launch. Every branch leaves evidence.

This is intentionally narrower than a full sandbox or a complete agent governance system. Charon does not claim to prove the model's intent, prevent host compromise, or make untrusted code safe by itself. It enforces a smaller invariant: an agent action should not cross into the machine boundary unless it satisfies policy, and the decision should be verifiable afterward.